

🔥 LEGION MANIFEST — System Inventory

```
> **Last verified:** 2026-07-05 08:28 EDT
> **Scope:** `C:\STUDIES_BACKUP\Legion-Jacked-Pipeline` + `C:\WEB CASE STUDY`
> **Method:** Direct file reads, regex sweeps, live Ray dashboard queries, MCP server introspection.
> **Author:** Ghost Rider 🧛 (Legion Leader's autonomous engine)
```

This is the ****single source of truth**** for what Legion is and where it lives. If something isn't in this document, it doesn't exist (or wasn't found).

⚡ EXECUTIVE SUMMARY

Metric	Count
Python files scanned (top 2 levels)	350+
Pydantic models	**31** (across 5 files)
Sovereign / DSP engines	**19** distinct engine classes
Ray actor classes	**31** <code>@ray.remote`</code> declarations
Ray tasks (functions)	12+ remote functions
FastAPI endpoints in <code>mcp_api_server.py`</code>	22
MCP servers wired in config	4 (2 active, 2 disabled)
Active LLM council	1 implicit (SovereignDiagnosticEngine routes Gemini → Ollama → deterministic)
Live Ray actors at time of audit	1 (<code>SwarmKnowledgeRegistry`</code>)
Path mismatches in MCP config	7
Missing venv	1 (<code>C:\WEB CASE STUDY\Scripts\python.exe`</code>)

1. PHYSICAL LAYOUT — THE TWO ROOTS

Root A: `C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\``

The ****architect's domain****. Where `mcp_api_server.py`` lives, where the LangGraph orchestrator is compiled, where the canonical Pydantic schemas sit.

****Key files:****

...

mcp_api_server.py	(60.4 KB) – The main FastAPI brain, 22 endpoints
api_server_v4.py	(3.9 KB) – "Brain" sub-server on port 8010
legion_graph.py	(25.1 KB) – Compiled LangGraph StateGraph
sovereign_engine.py	(23.7 KB) – Master diagnostic engine
sovereign_intelligence.py	(37.8 KB) – End-to-end diagnostic pipeline
sovereign_math_inference.py	(9.7 KB) – Math-inference audio gen
sovereign_schemas.py	(21.5 KB) – The Pydantic data spine
sovereign_chat.py	(25.1 KB) – Intent router + Ollama SQL gen
mcp_config.json	(7.6 KB) – Service definitions
legion_world_state.json	(0.3 KB) – Live world state

...

****Subdirectory:**** `ableton-session-intelligence\` (the ableton-intel lakehouse)

- `pydantic\_models.py` – 2 Pydantic models
- `lancedb\_web\_intel\_rag\` – `mined\_code\_vectors` table
- `E\_lancedb\_store\`, `lancedb\_highres\_audio\_rag\`,
`lancedb\_omni\_snowflake\_rag\` – vector stores
- `lakehouse\_data\`, `vectors\`, `data\` – data lake layers
- `notebooks\` (`.ipynb`) – Jupyter notebooks (15+)

Root B: `C:\WEB CASE STUDY\`

The ****producer's domain****. Where most of the ray actors, sonic DNA training, and creative tooling lives.

****Top-level gems (by size):****

...

Untitled-1.py	(187.7 KB) – Mega-notebook export (multiple actors)
upgraded_dynamic_batch_master_completenotebook.py	(127.1 KB) – Full VAE training notebook
upgraded_dynamic_batch_master_complete.py	(4.0 KB) – Companion script
mcp_api_server_old.py	(61.5 KB) – Older API server
mcp_api_server_2026_06_29.py	(66.0 KB) – Recent variant
mcp_api_server_2026_05_19.py	(24.3 KB) – Older variant
LINDS_WINDOWS_VOICE_APP_TEST.py	(32.1 KB) – Voice app

```
train_omni_v3_generation.py          ( 28.0 KB) - VAE trainer
legion_swarm_live_tools.py           ( 21.1 KB) - Swarm runtime
tools
adamscarmccoy-boop\                  (directory) - Boop project
Acoustic-DNA-Audio-Engine\            (directory) - Engine module
FretFlow-Audio-Engine\                (directory) - FretFlow module
Legion-Jacked-Pipeline\               (directory) - Mirror of Root A
Snoop_Stylizer_App\                  (directory) - Snoop styling app
sonic_dna_engine\                     (directory) - Sonic DNA engines
...
```

```
**Critical subdirectory: `sonic_dna_engine`** - neural audio mastering
stack:
```

```
...
sonic_dna_batch_master_v3.py          (older) - v3 master
sonic_dna_batch_master_v5_neural.py - v5 NEURAL (production)
sonic_dna_master_v2_train.py          - Trainer
sonic_dna_master_v3_train.py          - Trainer
sonic_dna_mini_test.py                - Mini test
sonic_dna_query.py                    - DNA query
audio_llm_agent.py                    - Audio LLM agent
audio_llm_train.py                    - Audio LLM trainer
sonic_dna_master_v3.pt                 - Master model weights
audio_llm_v1.pt                       - Audio LLM weights
...
---
```

2. PYDANTIC AGENTS - 31 MODELS, 5 FILES

```
### `mcp_api_server.py` - 21 API contract models
```

```
| Model | Purpose |
|---|---|
| `LangGraphRequest` | Workflow invocation |
| `SearchRequest` | Generic search payload |
| `LanceDBQuery` | Vector search request |
| `DuckDBQuery` | SQL query request |
| `AudioRequest` | Audio generation params |
| `TextInferenceRequest` | Phi LLM call |
| `VisionAnalyzeRequest` | Image analysis |
```

```

| `FileReadRequest` / `FileWriteRequest` | OS file ops |
| `ShellCommandRequest` | OS shell exec |
| `FindFilesRequest` | Glob search |
| `BrowserNavigateRequest` / `BrowserTabActionRequest` | Edge automation |
| `RunPythonFileRequest` / `TailLogRequest` | Script + log tooling |
| `DiscoverSchemaRequest` | DB introspection |
| `TaskCreateRequest` / `ToolExecuteRequest` / `SimpleChatRequest` | Task
router |
| `OpenAIRequest` | OpenRouter proxy |
| `FirebaseWriteRequest` | Firestore vault write |

### `sovereign_schemas.py` - 17 data spine models
`SovereignDAWDiagnostic`, `AudioTruth`, `SessionTime`,
`Lane1DuckDBAnalytics`, `Lane2LanceDBVectors`, `Lane3SystemLogs`,
`UserIntent`, `MarketScoreBreakdown`, `ScoredTrack`, `CatalogDNAProfile`,
`OraclePrediction`, `AlignedDSPTrackRecord`, `LiveTrackDelta`,
`MarketBenchmark`, `DiagnosticResult`, `MarketResult`, `DNAResult`,
`AnalysisRouter`, `SessionDNA`

### `legion_graph.py` - 1 stateful model
`AgentState` (L55) - LangGraph message-passing state container

### `sovereign_chat.py` - 1 model
`ChatRequest` (L485) - interface for `SovereignChat.chat()`

### `pydantic_models.py` (ableton-intel) - 2 models
`Lane1DuckDBAnalytics`, `CodeRoleResult`

---

## 3. SOVEREIGN ENGINES & DSP/INFERENCE STACK - 19 SYSTEMS

### Tier 1: Core Sovereign Stack
(`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline`)
| Engine | File | Role |
| ---| ---| ---|
| `SovereignEngine` | `sovereign_engine.py` L44 | Master diagnostic -
`analyze()`, `diagnose()`, `release_ready()`, `session_start()`,
`catalog_stats()`, `neighbors()` |

```

```

| `SovereignInferenceEngine` | `sovereign_math_inference.py` L12 |
Math-inference audio gen - `generate_from_kick()` |
| `SovereignIntelligencePipeline` | `sovereign_intelligence.py` L647 |
End-to-end orchestrator - `run_session()`, `run_catalog()` |
| `SovereignDiagnosticEngine` | `sovereign_intelligence.py` L399 |
Per-track diagnosis (Gemini → Ollama → deterministic) |
| `SovereignReportGenerator` | `sovereign_intelligence.py` L535 |
Markdown/JSON report emitter |
| `SovereignDeltaEngine` | `sovereign_intelligence.py` L242 | Live Δ
RMS/crest/dynamic-range engine |
| `SovereignChat` | `sovereign_chat.py` L298 | Intent router + Ollama SQL
gen |
| `AbletonSessionParser` | `sovereign_intelligence.py` L54 | `.als` XML →
tracks + FX chains |
| `DSPFeatureLookup` | `sovereign_intelligence.py` L155 | DuckDB feature
catalog |
| `VectorContextRetriever` | `sovereign_intelligence.py` L356 | LanceDB
RAG context |
| `AgentState` + LangGraph `app` | `legion_graph.py` L55, L464 | Compiled
StateGraph orchestrator |
| `api_server_v4.py` | `api_server_v4.py` | The "Brain" sub-server (port
8010) |

```

Tier 2: Neural Audio Pipeline (`C:\WEB CASE STUDY\sonic\_dna\_engine`)

```

| Engine | File | Role |
|---|---|---|
| `SonicDNAMaster` v3 | `sonic_dna_batch_master_v3.py` | Mastering net
wrapper |
| `SonicDNAMaster` v5 NEURAL | `sonic_dna_batch_master_v5_neural.py` |
**Current production** |
| `MasteringNet` / `MasteringTrainer` | `sonic_dna_master_v3_train.py`,
`v2_train.py` | PyTorch training side |
| `SonicDNA` + `SonicDNATrainer` | `sonic_dna_mini_test.py` | DNA
extractor |
| `AudioLLM` | `audio_llm_agent.py` + `audio_llm_train.py` |
Audio-language model agent |

```

Tier 3: Forest / Swarm / Orchestrator (`C:\WEB CASE STUDY`)

```

| Engine | File | Role |
|---|---|---|

```

```
| `ForestEngine` cell | `forest_engine_cell.py` | Distributed code-forest
|
| `EnterpriseForestEngine` | `enterprise_forest_engine.py` |
Enterprise-scale forest |
| `UnifiedForest` runner | `unified_forest_run.py` | Forest orchestrator |
| `SovereignConductor` | `sovereign_conductor.py` | Multi-engine conductor
|
| `SovereignAudioLink` / Hardened | `sovereign_audio_link.py`,
`sovereign_hardened_link.py` | Audio I/O bridge |
| `CodeForestEngine` | `code_forest_engine.py` | Code-aware forest |
| `LegionSonicEngine` | `legion_sonic_engine.py` + orchestrator | Sonic
orchestrator pair |
| `LegionSwarmLiveTools` | `legion_swarm_live_tools.py` | Live swarm tool
runtime |
| `SocialEngine` | `social_engine.py` | Outreach/social layer |
```

Tier 4: RAG / Code Intel

```
| Engine | File | Role |
|---|---|---|
| `LegionLakehouse_RAG` (FastMCP) | `mcp_rag_server.py` | MCP RAG server
(port 8003), `semantic_code_search()` |
| `mcp_api_server.py` | `mcp_api_server.py` | Main FastAPI: 22 endpoints,
21 Pydantic models |
```

4. RAY ACTOR FLEET — 31 `@ray.remote` CLASSES

Cluster Status (live at audit time)

```
- **gcs_server**: PID 8928 
- **raylet**: PID 20812 
- **Dashboard**: `http://127.0.0.1:8265` 
- **Live actors**: 1 (`SwarmKnowledgeRegistry`, namespace `legion`)
- **Recent jobs**: 3 (1 RUNNING, 2 SUCCEEDED)
```

Active / Runtime-critical

```
| # | Actor | File | Purpose |
|---|---|---|---|
| 1 | `SwarmKnowledgeRegistry`  LIVE | `ray_arrow_swarm.py` | Global
in-memory store of PyArrow tables. The one alive right now. |
```

```

| 2 | `CodeSwarmKnowledgeRegistry` | `ray_code_swarm.py` | Twin for
code-notebook knowledge. |
| 3 | `SwarmKnowledgeWorker` | `swarm_knowledge_daemon.py` | Background
daemon pushing tables. |
| 4 | `RegistryClient` | `mix_audit_agent.py` | Client wrapper for the
registry. |

### Training actors
| # | Actor | File | Purpose |
|---|---|---|---|
| 5 | `TrainerActor` | `omni_v3_train.py` | Trains OmniCondVAE
(1036→512→256→128→32). Per-shard. |
| 6 | `TrainerActor` | `train_omni_v3_generation.py` | Generation-side
training loop. |
| 7 | `TrainerActor` (×3) |
`upgraded_dynamic_batch_master_completenotebook.py`, `Untitled-1.py` (×2)
| Notebook copies. |
| 8 | `VAETrainer` | `upgraded_dynamic_batch_master_completenotebook.py` |
VAE-specific trainer. |
| 9 | `VAETrainer` | `Untitled-1.py` | Notebook twin. |
| 10 | `GenerationWorker` | `fretflow_ray_gen.py` | **The big one.** Loads
`fretflow_omni_v3.pt`, runs encoder→latent→decoder. |
| 11 | `EmbedWorker` | `run_vector_rebuild.py` | LM-Studio embedding
worker (1024-D Snowflake). 6 workers parallel. |

### Cognitive / Omni-cell
| # | Actor | File | Purpose |
|---|---|---|---|
| 12 | `GenomeBrain` | `cell_30_clean.py` | Genome transformer brain.
References live SwarmKnowledgeRegistry. |
| 13 | `GenomeBrain` | `upgraded_dynamic_batch_master_completenotebook.py`
| Notebook twin. |
| 14 | `GenomeActor` | `Untitled-1.py` | Per-track genome inference. |
| 15 | `OmniCognitiveWorker` | `cell_47_clean.py`,
`dual_vector_pipeline.py`, `openai_parquet_god_cell.py` | 13-D audio +
1024-D text dual-vector brain. |
| 16 | `OmniKnowledgeWorker` | `Untitled-1.py` | Knowledge-side of omni
worker. |
| 17 | `TensorEvaluatorActor` | `Untitled-1.py` | Per-shard tensor
metrics. |

```

```
| 18 | `SovereignInferenceActor` | `Untitled-1.py` | Sovereign inference  
inside Ray shard. |
```

Audio / DSP

```
| # | Actor | File | Purpose |  
|---|---|---|---|  
| 19 | `DSPAlignmentActor` | `dsp_alignment_actor.py` | **Critical.**  
scikit-learn StandardScaler + LanceDB feature matching. **One per CPU  
core.** |  
| 20 | `DSPAlignmentActor` | `dsp_alignment_actor_local.py` | Local-mode  
twin. |  
| 21 | `DSPAlignmentActor` | `run_fire_test_standalone.py` | Standalone  
test twin. |  
| 22 | `MyActor` | `ray_swarm_test.py` | Test actor – generic. |
```

Indexing / RAG

```
| # | Actor | File | Purpose |  
|---|---|---|---|  
| 23 | `OllamaEmbeddingWorker` | `ollama_rag_indexer.py` |  
Pydantic-validated chat-memory indexer. |  
| 24 | `IntelligenceBridge` | `intelligence_bridge.py` | The "Warden's  
ground-truth" actor. Industry baselines. |  
| 25 | `IndependentWorkFinderActor` | `indie_work_finder_actor.py` |  
5-stage outreach lead pipeline. |
```

Engine / Core

```
| # | Actor | File | Purpose |  
|---|---|---|---|  
| 26 | `SovereignEngine` (Ray-wrapped) | `sovereign_engine.py` | Master  
diagnostic engine as Ray actor. |  
| 27 | `GenomeTransformer` + `GenomePolicy` | `cell_30_clean.py` | Helpers  
wrapped by `GenomeBrain`. |
```

Remote tasks (functions, not classes)

```
- `ingest_and_convert_json` (`ray_arrow_swarm.py`) – JSON→Arrow parallel  
ingestion  
- `categorize_samples` / `categorize_streaming` – Ray-parallel sample  
categorization  
- `crawl_and_vectorize_all_ray` – document crawler  
- `search_code_swarm` – code RAG search
```

```
- `fire_test_` (multiple) - diagnostic fire-tests
- `download_ray_docs` - Ray doc crawler
```

```
---
```

5. MCP SERVER CONFIG (current state)

```
```json
```

```
{
 "mcpServers": {
 "google-cloud-resource-manager": { "disabled": true },
 "google-developer-knowledge": { "disabled": true },
 "legion-architect": { "active": true, "command":
"c:\\STUDIES_BACKUP\\.venv_fresh\\Scripts\\python.exe" },
 "legion-mcp-rag-server": { "active": true, "command":
"c:\\WEB CASE STUDY\\Scripts\\python.exe" }
 }
}
```

```
```
```

```
---
```

6. KNOWN ISSUES & GAPS

Critical

| # | Issue | Impact |
|---|--|---|
| 1 | `C:\\WEB CASE STUDY\\Scripts\\python.exe` **MISSING** | RAG server venv broken; needs `python -m venv` recreate. |
| 2 | `.venv_fresh` (used by `legion-architect`) does NOT have `ray` installed | `mcp_api_server.py` cannot query the Ray cluster. |
| 3 | `mcp_api_server.py` sys.path does NOT include `C:\\WEB CASE STUDY` | 6 sovereign/forest files at wrong path → import failures. |
| 4 | `legion-architect` MCP server uses `.venv_fresh`; Ray jobs use `Python312` | Driver/runtime interpreter mismatch. |

Quality

| # | Issue | Impact |
|-----|-------|--------|
| --- | --- | --- |

```
| 5 | `Untitled-1.py` is 187.7 KB | Notebook export, needs refactor into modules. |
```

```
| 6 | 3 copies of `mcp_api_server.py` (`mcp_api_server.py`,  
`mcp_api_server_2026_05_19.py`, `mcp_api_server_2026_06_29.py`,  
`mcp_api_server_old.py`) | Drift risk; consolidate. |
```

```
| 7 | `mcp_rag_server.py` table path differs from MCP config default |  
Path resolution in RAG server hardcoded to a different LANCEDB_PATH. |
```

🟢 Working as designed

- LangGraph `AgentState` orchestrator compiles
- DuckDB path resolves correctly
- LanceDB at `C:\STUDIES\_BACKUP\vectors\lancedb\_store` exists
- Ray cluster is healthy (gcs + raylet + dashboard)
- Firebase Admin SDK initializes with `sa-key-new.json`
- Sonic DNA v5 NEURAL pipeline functional

7. PORT MAP (services Legion touches)

| Port | Service | Status |
|---------------------|---------------------------------------|---|
| --- --- --- | | |
| **8001** | `mcp_api_server` (legion-architect) | Configured |
| **8003** | `legion-mcp-rag-server` (FastMCP SSE) | Configured |
| **8010** | `api_server_v4` (Brain) | Spawned on-demand via
`/brain/start` |
| **8265** | Ray dashboard | ✅ Live |
| **10001** | Ray client server | ✅ Live |
| **6379** | Ray internal gcs | ✅ Live (Ray gcs_server) |
| **9222** | Edge browser remote debug | Used for `browser/tabs` |
| **11434** | Ollama default | Used by Phi3 inference |
| **1234** | LM Studio default | Used by Snowflake embed |
| `http://100.113.76` | | Tailscale Ollama on remote GPU Configured |

8. DATA LAKE LAYOUT

DuckDB databases

- `C:\STUDIES\_BACKUP\data\metadata\sonic\_core\_v2.duckdb` — main catalog

```
- `C:\WEB CASE STUDY\web_intel_sonicdb.duckdb` - web intel sonicdb
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\web
_intel_sonicdb.duckdb` - twin

### LanceDB stores
- `C:\STUDIES_BACKUP\vectors\lancedb_store` - main vector store
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\lan
cedb_web_intel_rag` - RAG code
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\lan
cedb_highres_audio_rag` - high-res audio
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\lan
cedb_omni_snowflake_rag` - omni Snowflake
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\E_1
lancedb_store` - E: mirror
- `C:\WEB CASE STUDY\lancedb_memory` - chat memory
- `C:\WEB CASE STUDY\lancedb_store` - workspace store
- `C:\WEB CASE STUDY\lancedb_data` - workspace data
- `C:\WEB CASE STUDY\rag-v1` - RAG v1
- `C:\STUDIES_BACKUP\vectors\lancedb_store` - vectors

### Model weights (ONNX / PyTorch)
- `C:\WEB CASE STUDY\dna_brain.onnx` (5.3 KB)
- `C:\WEB CASE STUDY\fretflow_omni_v4.onnx` (3.9 KB)
- `C:\WEB CASE STUDY\omni_master_brain_v1.onnx` (8.2 KB)
- `C:\WEB CASE STUDY\real_data_brain.onnx` (5.1 KB)
- `C:\WEB CASE STUDY\sonic_dna_engine\sonic_dna_master_v3.pt`
- `C:\WEB CASE STUDY\sonic_dna_engine\audio_llm_v1.pt`
-
`C:\STUDIES_BACKUP\Legion-Jacked-Pipeline\ableton-session-intelligence\bes
t_sonic_dna_pro.pt`

---

## 9. RECENT SESSION REPORTS
```

```
- `LEGION_SESSION_REPORT_2026-06-25.md` - last big session
- `INVENTORY_20260622_222600.txt` - June 22 inventory (empty)
- `legion_pipeline_whitepaper.md` - pipeline architecture
- `VIBE_ENGINE_WHITEPAPER.md` - vibe engine design
```

10. LEGION'S OPERATIONAL CONTRACT

Legion is a ****multi-tier cognitive architecture**** with three brain hemispheres:

1. ****Producer**** (Root B) - trains, generates, indexes, fires tests
2. ****Architect**** (Root A) - orchestrates, queries, serves, reasons
3. ****Bridge**** (Ray fleet) - distributes, paralllelizes, scales

All three are wired together by:

- ****Pydantic**** for the data contract firewall
- ****LanceDB**** for semantic memory
- ****DuckDB**** for analytical truth
- ****Ray**** for distributed compute
- ****FastAPI/MCP**** for service exposure
- ****LangGraph**** for agentic orchestration
- ****Ollama/LM Studio**** for local LLM

****🔥 END OF MANIFEST - Updated by Ghost Rider on 2026-07-05 08:28 EDT****